Lógica de Programação e Estrutura de Dados

Unidade Curricular I

Senac

Iconografia

Conheça a iconografia utilizada no curso. Sempre que você vir um desses ícones, redobre sua atenção. Em alguns casos, você deverá clicá-lo para visualizar a informação correspondente.

Atenção!
Este ícone assinala/destaca para você uma informação de extrema importância.

Agora é com você!
Esse é o momento reservado para que você registre seus comentários sobre alguma atividade e/ou reflexões propostas.

O que é?
Significado de palavras utilizadas no texto, visando facilitar sua compreensão.

Saiba mais
Apresenta informações que complementam o texto e auxilia a reflexão sobre o assunto estudado.

Exemplificando
Indica um exemplo ou uma situação (case) para que você entenda melhor o assunto tratado.

Legislação
Apresenta citações de documentos legais.

Lembrete
Este ícone indica uma informação que merece ser destacada.

Dica
Sempre que este ícone aparece, sugere alguma prática que vai ajudar seu trabalho no dia a dia.

Pausa para refletir
Questões para reflexão. Não há exigência de comentários feitos pelo participante.

Relembrando
Sempre que este ícone aparece, sugere a recuperação de ideias, conceitos e assuntos já vistos.

Biblioteca Virtual
Este ícone indica leitura na biblioteca virtual disponível no Avas.

Momento distração
Sempre que este ícone aparece, sugere alguma leitura para a distração.

Participe
Sugere que você participe mais, contribuindo nos fóruns de discussão.

Vale a pena assistir
Sugere que você assista a um filme ou entretenimento.

Índice

Abertura — Introdução 04

Capítulo 1 - Definição de algoritmos 06

- Definições de hardware, software e programas 06
- O que é um Algoritmo 06
- Depuração de erros 07

Capítulo 2 - Técnicas de representação de algoritmos 10

- Representação de algoritmos 10

Capítulo 3 - Produção de algoritmos 13

- Algoritmo 13
- Componentes do algoritmo ..... 14 e 16
- Componentes do algoritmo - Constantes 14
- Componentes do algoritmo - Variáveis 15
- Principais comandos de um algoritmo 17

Capítulo 4 - Estruturas de algoritmos 20

- Estruturas de algoritmos 20

Capítulo 5 - Modularização de algoritmos 28

- Modularização de algoritmos 28

Capítulo 6 - Recursividade 30

- Função fatorial 31
- Sequência de Fibonacci 33
- Problema da Torre de Hanói 34

Capítulo 7 - Estruturas de dados 37

- Estrutura de dados 37
- Estrutura de dados homogênea 37
- Estrutura de dados heterogênea - Registros 40
- Estrutura de dados heterogênea - Trabalho com registros 41
- Estrutura de dados heterogênea - Arquivos 42

Capítulo 8 - Linguagens estruturadas 43

- Linguagens estruturadas 43
- Síntese da aula 44
- Conclusão 44

Nesta unidade trataremos dos seguintes temas:

- Lógica de programação;
- Fluxograma;
- Algoritmos;
- Estruturas sequenciais, condicionais e repetitivas;
- Funções e recursividade;
- Estrutura de dados homogênea e heterogênea;
- Linguagens estruturadas.

Ao final desse estudo, você terá construído as seguintes competências:

- Solucionar problemas de navegação por meio de algoritmos;
- Transformar layout em código, mediante o uso de ferramentas apropriadas;
- Desenvolver lógica de programação para solução de problemas;
- Codificar soluções utilizando uma linguagem estruturada.

Introdução

No nosso dia a dia, nos deparamos com várias situações onde precisamos chegar a um objetivo específico aplicando um conjunto finito de passos.

Por exemplo, para trocar uma lâmpada queimada, precisamos comprar uma lâmpada nova, desligar a chave geral (para não haver riscos de choque elétrico), pegar uma escada, tirar a lâmpada queimada, colocar a nova, ligar a chave geral e ligar o interruptor da lâmpada.

Vejam que o objetivo era trocar uma lâmpada e para isso traçamos passos para chegar ao resultado final. Bom, temos vários outros exemplos em nossa vida, como chegar ao nosso trabalho pegando um melhor caminho, fazer um bolo, fazer uma faxina, organizar os livros etc.

Introdução

Podemos dizer que o objetivo principal da lógica de programação é demonstrar técnicas para resolução de problemas e consequentemente automatização de tarefas, sendo o seu aprendizado essencial para formação de um bom programador, servindo como base para o aprendizado de todas as linguagens de programação, estruturadas ou não.

Vamos ver alguns conceitos importantes sobre lógica:

Lógica pode ser definida como sendo o estudo das leis do raciocínio e do modo de aplicá-las corretamente na demonstração da verdade.

A utilização da lógica na vida do indivíduo é constante, visto que é ela quem possibilita a ordenação do pensamento humano.

A lógica de programação consiste em aprender a pensar na mesma sequência de execução dos programas de computador.

?

Lógica de programação é a técnica de ordenar pensamentos para atingir um determinado objetivo. Esses pensamentos podem ser descritos como uma sequência de instruções, que devem ser seguidas para se cumprir uma determinada tarefa.

Sequência Lógica são passos executados até atingir um objetivo ou solução de um problema. Assim, a lógica de programação permite definir a sequência lógica para o desenvolvimento de sistemas e programas de computador.

Antes de estudarmos os algoritmos, se fazem necessários conhecimentos prévios sobre hardware, software e programas.

Você sabe o que são?

?

Podemos definir hardware como a parte física do computador, ou seja, é tudo aquilo que podemos tocar, que é palpável. Exemplo: teclado, monitor de vídeo, impressora etc.

O software é a parte lógica do computador, ou seja, é tudo aquilo que não podemos tocar, que é abstrato. Exemplo: Programas.

Os programas são conjuntos de instruções (comandos) logicamente organizados que devem ser processados para resolver um determinado problema.

Vamos conhecer as etapas para a criação de um programa?

- Definição do programa
- Criação do algoritmo
- Uso de recursos gráficos
- Codificação da solução
- Teste de programa
- Depuração de erros
- Documentação da solução

O que é um Algoritmo

O que é necessário saber antes de usar o computador para codificar o programa?

Primeiramente, você deve conhecer qual é o problema que se deseja resolver e caracterizálo de maneira clara e completa, normalmente descrita por meio de um algoritmo.

Aliás, você sabe o que é um algoritmo?

?

Um Algoritmo é uma sequência de instruções ordenadas de forma lógica para a resolução de uma determinada tarefa ou problema.

Essa capacidade de resolução de problemas é também transportada para a construção de programas de computador, ou seja, por meio de algoritmos.

O conceito de algoritmo não é exclusividade da área da computação. Um algoritmo não representa, necessariamente, um programa de computador e sim os passos necessários para realizar uma tarefa ou solucionar um problema, seja de que área for.

Nos deparamos com vários problemas no nosso cotidiano que exigem soluções e podemos usar algoritmos para resolvê-los.

Um algoritmo é uma sequência finita e verificável de ações que escrevem como um problema pode ser definido. Sendo assim, ele deve:

- Ter início e fim.
- Ser descrito em termos de ação, não ambíguas e bem definidas.
- Que as ações sigam uma sequência ordenada.

Podemos citar como exemplos de algoritmos:

- Instrução de operação de equipamentos;
- Uma receita culinária;
- Instruções de montagem;
- Partituras de música;
- Programa de computador.

Depuração de erros

E como fazemos para verificar os resultados do programa criado pelo algoritmo?

Submetemos uma massa de dados a testes, onde os resultados são comparados e verificados. Na existência de erros, o programa é corrigido e compilado novamente.

Depuração de erros

Uma das técnicas mais utilizadas na depuração de erros é o tabuleiro chinês, onde é confeccionada uma planilha com todas as variáveis e os possíveis valores de cada uma são preenchidos levando em consideração a execução lógica do algoritmo e atribuição de valores a cada um dos elementos que o compõe, conforme é demostrado no algoritmo de teste 1 na ilustração abaixo.

Algoritmo de teste 1

Algoritmo

```
{Declarações}  ⇒  comentários
Const  ⇒  constante
        A=3

Var  ⇒  variável
      Inteiro: B,C, D, Resto, Quociente
      Lógico: ACHOU, TEM, COMPROU        ⇒  Verdadeiro ou Falso; 1 ou 0
      Real: Média, Soma

INÍCIO

      B < -1
      B < -B + 1
      IMPRIMA          ⇒  Palavras sublinhadas são reservadas
      c < -A + B
      c < -A * C
      IMPRIMA (D, C)
      ACHOU < - (B >A)
      Resto < - 30 mod 4
      Quociente < - 20 div 6
      TEM < - FALSO
      COMPROU < - não TEM
      C < - Quociente + (A**B)
      D < - 2
      IMPRIMA ("C", C, "D", D)
      LEIA (B, C, D)
      IMPRIMA (B, C, (D+A), (A**B))
      Soma < - A+B+C+D
      Média <- Soma/4
FIM
```

Variáveis	Valor	2º valor	3º valor	4º valor
B	1	2	4	
C	5	14	6	
D	15	2	3	
Resto	2			
Quociente	5			
ACHOU	FALSO			
TEM	FALSO			
COMPROU	VERDADEIRO			
Media	4			
Soma	16			

Para que você compreenda melhor a depuração de dados, se faz necessário o conhecimento prévio de algumas definições, como:

O que é dado ?

É tudo aquilo que vai ser processado. É a parte integrante da informação, ou seja, é a matéria-prima que gera a informação.

A = 10
B = 2
Some **A** + **B** e coloque em **C** ⇒ instrução
Resposta: C = 12

Observe que C = 12 é uma informação, porque é o resultado de **A+B**, então tem-se que dado é o que entra e informação é o que sai, ou seja, é o resultado do processamento.

O que é uma informação ?

É todo o dado significativo, disponível em um formato adequado ao seu uso.

O que é dado ?

É toda atividade, na qual dados iniciais são submetidos a um processamento qualquer, gerando informações desejadas.

Encerramento do
Capítulo 1 - Definição de Algoritmos

Os algoritmos podem ser representados de três formas diferentes:

Descrição narrativa

Em uma linguagem técnica (português ou inglês) e é utilizado em receitas, instruções etc.

A textual não é a melhor opção para representação de algoritmos porque ela pode gerar interpretações erradas, ambiguidades ou imprecisões. Por exemplo, vejamos a frase:

Samara está estudando para o Senado.

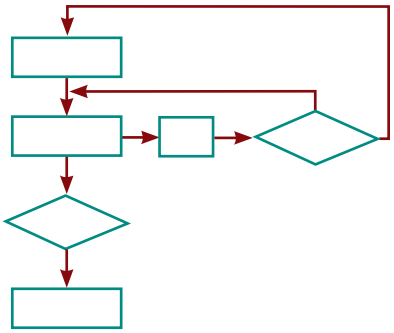
A partir da frase acima, podemos chegar, pelo menos, a duas conclusões:

- A Samara está estudando para ser uma política e concorrer para ser senadora; ou
- A Samara está estudando para o concurso do Senado.

A segunda conclusão é no nosso contexto, não é? Bom, com apenas essa descrição bem informal, não temos certeza nenhuma do que ela quer dizer se não estivermos no mesmo contexto de quem a descreveu.

Recursos gráficos

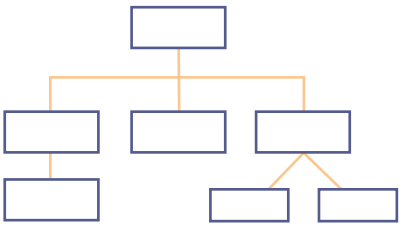
Fluxograma



Os fluxogramas nada mais são do que representações gráficas da lógica a ser seguida por um programa ou sistema.

Visualmente, após um entendimento do que cada forma geométrica significa, fica fácil entender o que propõe o algoritmo representado por um fluxograma, porém dá um trabalho para mantê-lo porque uma única correção pode requerer uma reorganização de muitas instruções.

Método de Jackson



JSP (Jackson Structured Programming) foi criado por Michael A. Jackson (1970). É um método para elaboração de programas como composições de processos sequenciais.

O diagrama do JSP é utilizado para explicar o funcionamento interno do programa, suas entradas e saídas de dados e seus componentes.

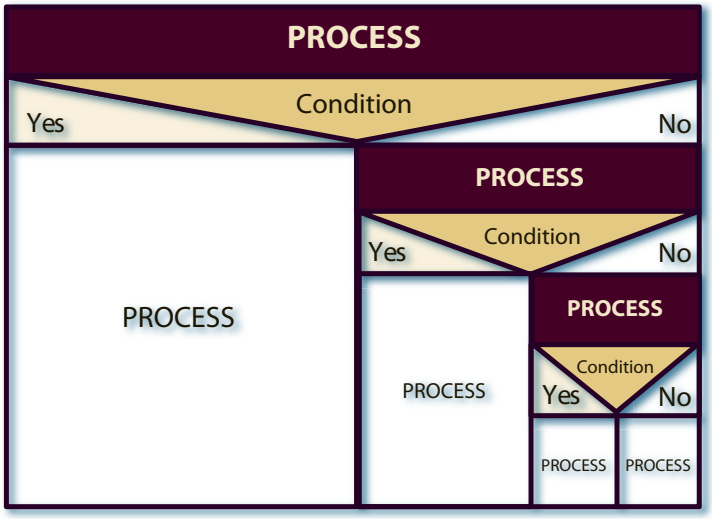
Ele é um método para programação estruturada, com base no fluxo de dados e na estrutura do programa. O diagrama é lido de forma Top-Down e da esquerda para direita.

Diagrama de Nassi

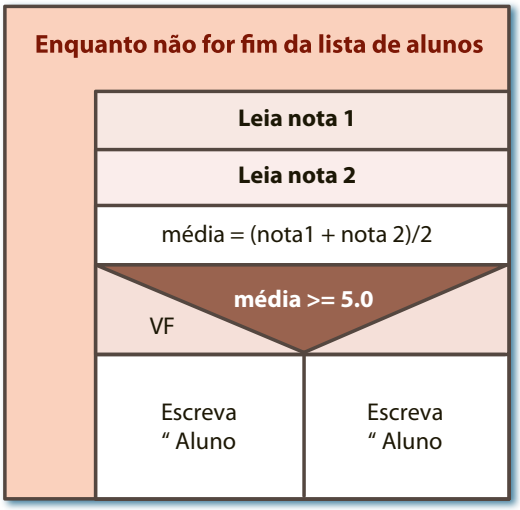
Também conhecido como diagrama Chapin, esta ferramenta de representação oferece grande clareza para a representação de sequenciação, seleção e repetição num algoritmo, utilizando-se de uma simbologia própria.

A ideia básica deste diagrama é representar as ações de um algoritmo dentro de um único retângulo, subdividindo-o em retângulos menores, que representam os diferentes blocos de sequência de ações do algoritmo. Seleção e repetição também são representadas de forma gráfica, dentro dos retângulos. (Figura A)

Embora os diagramas N-S ofereçam uma representação muito clara do algoritmo, à medida que os algoritmos vão se tornando mais complexos, fica difícil realizar os desenhos necessários numa única página, prejudicando a sua visualização. (Figura B)



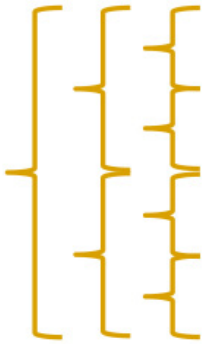
(Figura A)



(Figura B)

Diagrama de Warnier-or

Um diagrama Warnier-or representa graficamente a estrutura hierárquica de um programa, de um sistema ou de dados. É desenhado horizontalmente na página, ao invés de seguir para baixo com blocos.



Cada chave no diagrama representa um quebra funcional do item mencionado na sua parte superior. Além de mostrar a estrutura hierárquica, também ilustra o fluxo de controle através da estrutura.

Os quadros Warnier-or podem representar as estruturas de dados, relatórios ou estruturas de programas.

Linguagem algorítmica

Em uma linguagem de programação (Pascal, C#, Java, VB etc.), utilizada diretamente por programadores mais experientes.

A linguagem algorítmica, também chamada de pseudocódigo, portugol ou pseudolinguagem, é a mais utilizada por ser mais formal do que a descrição narrativa e mais fácil de manter do que um fluxograma.

Além disso, essa linguagem é menos rígida do que uma linguagem de programação, dando assim liberdade ao programador para "rascunhar" seus futuros programas sem ficar engessado na rigidez da linguagem de programação escolhida por ele.

Veja abaixo um exemplo de uma estrutura de pseudocódigo:

Pseudocódigo

```
leia (x,y)          {Esta linha é um comentário}
se X > y então
        escreva ("x é maior")
senão
        se y > x então
                escreva ("y é maior")
        senão
                escreva ("x e y é são
                iguais")
        fim-se
fim-se
```

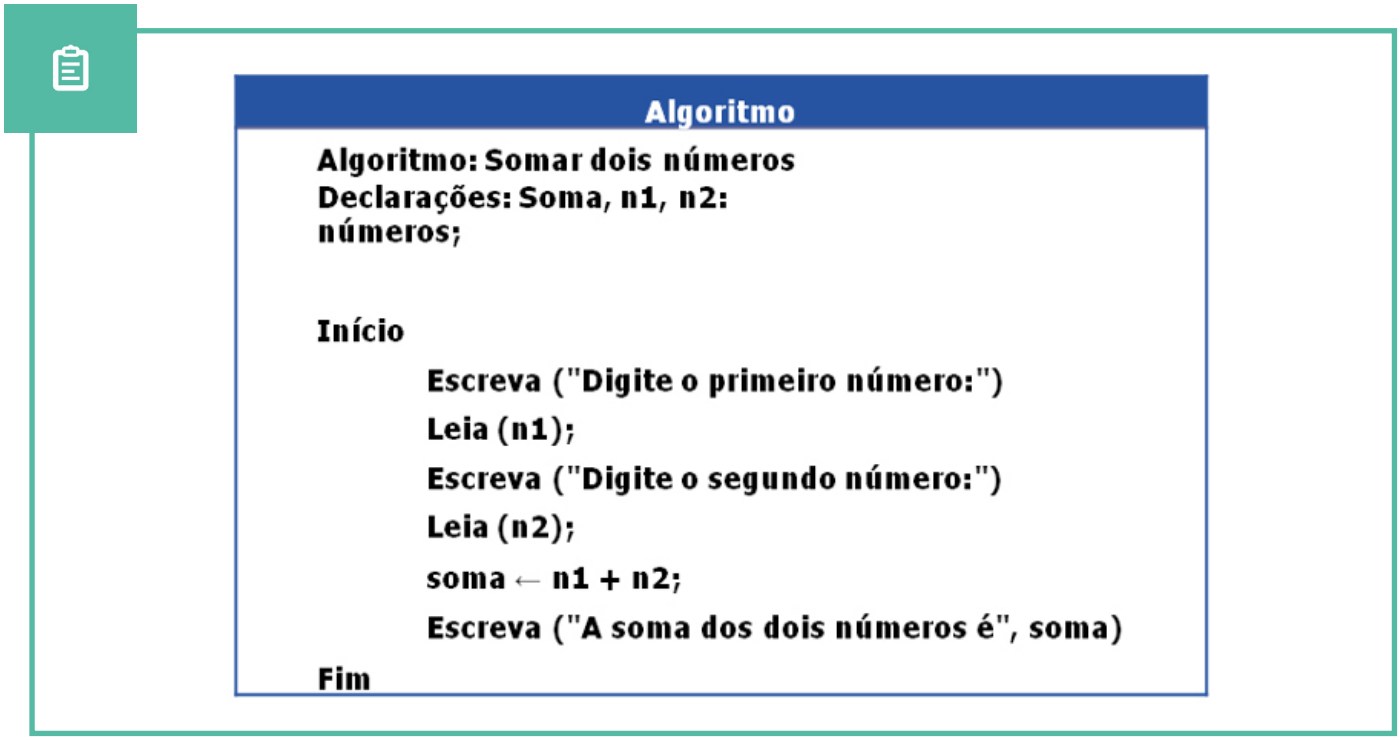
Encerramento do
Capítulo 2 - Ténicas de Representação de Algoritmos

A produção de algoritmo segue algumas regras. Como deve ser constituída a estrutura de um algoritmo?

Sua estrutura é composta do nome do algoritmo, área de declaração de variáveis e bloco de comandos, demarcado com as palavras Início e Fim.

Vamos ver como seria isso?

algoritmo nomealgoritmo;
//área de declaração de constantes e variáveis
//constantes
//variáveis
inicio
//área de entrada - processamento - saída
//todos os comandos do algoritmo
fim.



Algoritmo

```
Algoritmo: Somar dois números
Declarações: Soma, n1, n2:
números;

Início
        Escreva ("Digite o primeiro número:")
        Leia (n1);
        Escreva ("Digite o segundo número:")
        Leia (n2);
        soma ← n1 + n2;
        Escreva ("A soma dos dois números é", soma)
Fim
```

Componentes do algoritmo

A implementação de um algoritmo faz uso de espaços de memória chamados variáveis, onde são armazenados todos os dados que são imputados durante a entrada de dados, para serem processados e consequentemente gerar uma saída de dados, objetivando a solução de um problema.

As variáveis que compõem um algoritmo podem ser constantes ou variáveis de acordo com o seu comportamento ao longo da execução do algoritmo.

CONSTANTES

É um determinado valor que não se altera ao longo do tempo durante a execução do programa.

As constantes podem ser:
- Numéricas;
- Literais;
- Lógicas.

VARIÁVEIS

São posições de memória principal cujo conteúdo varia da execução do programa.

As variáveis podem ser:
- Numéricas;
- Literais;
- Lógicas.

Componentes do algoritmo - Constantes

Vamos conhecer um pouco mais sobre os tipos de constantes.

CONSTANTES NUMÉRICAS

A representação de uma constante nos algoritmos é feita no sistema decimal, podendo ser um número com ou sem parte fracionária. Como, por exemplo: 365; -40; +2; 3,1; 0,2 X 10.

CONSTANTES LITERAIS

Aparecem sempre entre aspas (" "). Como por exemplo: "Maria"; "365"; "Rua Paulo, 136"; "CEP.26.236-110"; "-1".

CONSTANTES LÓGICAS

Assumem somente dois valores: FALSO e VERDADEIRO.

!

As constantes numéricas podem ser positivas ou negativas, de acordo com o sinal que precede os algarismos formadores do número.

Componentes do algoritmo - Variáveis

VARIÁVEIS NUMÉRICAS

Só se atribui valores numéricos. Como por exemplo: MATR = 400; MÊS = 12; SOMA = 100.

VARIÁVEIS LITERAIS

Mensagem (“ “). Como por exemplo: NOME = “Ana Angélica”; END = “Rua Eva, 20”; Z = “54”.

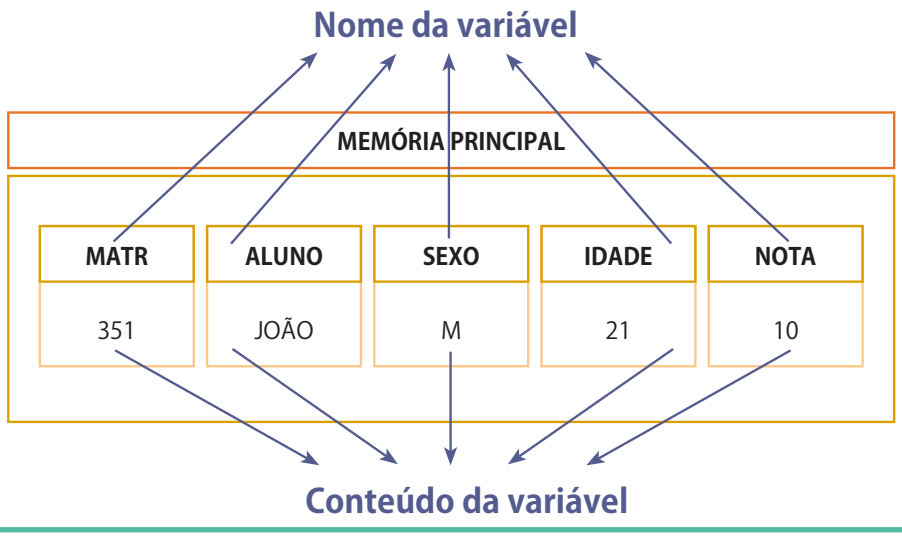
VARIÁVEIS LÓGICAS

Os valores são falso ou verdadeiro. Como por exemplo: OK = Falso; TUDO = Verdadeiro; Y = Falso.

Exemplo:

MATR = 351 ALUNO = “JOÃO” SEXO = “M” IDADE = 21 NOTA = 10

A figura abaixo ilustra o comportamento das variáveis na memória principal.



Mas, como são criadas essas variáveis?

Tanto para a criação quanto para a declaração de variáveis, devemos seguir algumas regras. Observe os exemplos de criação e declaração de variáveis a seguir:

Criação de Variáveis

DESCRIÇÃO	VARIÁVEIS VÁLIDAS	VARIÁVEIS INVÁLIDAS
NOME DO ALUNOI	NOME	NOME DO ALUNO
TOTAL	TOT1	TOT/3
MÉDIO DO ALUNO	MEDIALU	SOM-1
PRIMEIRA NOTA	NOTA1	NOTA DO ALUNO
TOTAL DA EMPRESA	TOTALDAEMPRESA	TOTAL DA EMPRESA

Declaração de Variáveis

Declaração de variável	Tipo de variável		
	exemplo		
Declare VAR tipo	Declare	NOME	literal
Onde:	Declare	NOTA	numérico
Declare ⇒ palavra-chave	Declare	TESTE, Z	lógica
VAR ⇒ nome da variável	Declare	ANO, PRECO	numérica
Tipo: ⇒ tipo da variável (numérica, literal ou lógica)			

Criação de variáveis

As variáveis devem ser escritas em letras maiúsculas.

✓ Podem ser usadas letras e algarismos, porém necessariamente deve começar por letra; não podem ser usados caracteres especiais.

✓ Devem possuir a menor quantidade possível de caracteres.

✓ O nome da variável deve lembrar o seu conteúdo.

Criação de variáveis

✓ No início do programa, as variáveis são declaradas de acordo com o seu tipo.

Principais comandos de um algoritmo

Existem vários tipos de comandos que podem ser usados em um algoritmo.

Comando de atribuição (←)

Usado para atribuir valores às variáveis.

Sintaxe:

VAR f <argumento>

Onde: VAR : variável; f: comando de atribuição; <argumento>: pode ser uma variável ou uma constante.

Podemos citar como exemplo: NOME f "Ary"; SALMIN f 70,00; OK f Falso.

Para uma variável declarada como número só poderão ser atribuídos valores numéricos. O mesmo ocorre em relação às variáveis do tipo literais e lógicas.

Podemos usar operadores que podem ser:

Operadores aritméticos e funções

Permite que se façam operações com variáveis, constantes e valores inteiros ou reais.

+ Soma **- Subtração** *** Multiplicação**

/ Divisão **** Potenciação**

m mod i: Resulta no resto da divisão inteira de m por i
m div i: Resulta no quociente da divisão de m por i

As funções mais comuns da matemática são definidas e válidas no nosso curso:
Log (x) - Logaritmo da base 10 de x
Sen(x) - Seno de x
Abs(x) - Valor absoluto de x
Raiz(x) - Raiz quadrada de x

Operadores relacionais

São operadores utilizados com 2 operadores, que podem ser expressos em valores, constantes ou variáveis. Têm como resultado um valor lógico (verdadeiro "1" ou falso "0")

= igual **<> diferente** **> maior** **< menos**

>= maior ou igual **<= menor ou igual**

Para as variáveis A, B do tipo inteiro, responda para A <- 5 e B <- 3

Expressão	Resultado
A=B	Falso
A<>B	Verdadeiro
A>B	Verdadeiro
A<B	Falso
A>=B	Verdadeiro
A<=B	Falso

Operadores lógicos

São operadores utilizados como operando do tipo lógico (podem ser expressões relacionais ou variáveis do tipo lógico) e dão um resultado do tipo lógico.

São eles:

e ("and") : ^ conjunção **ou ("or") :** v disjunção **não ("not") :** ¬ negação

Acesse a biblioteca virtual e saiba mais sobre operadores lógicos, estudando a tabela verdade, por meio da leitura do documento: "Tabela_verdade.pdf ".

Comando de entrada (leia)

Para que possamos obter dados do meio exterior para uso do computador (memória principal), estes têm que vir por meio de comandos de dispositivo de entrada.

O comando de entrada é usado para entrar com dados para o processamento, ou seja, ao digitarmos um valor para uma determinada variável, o processador irá **LER** o valor e armazenar na variável determinada.

Sintaxe:

leia < var >

Onde

leia : comando de entrada
< var > : variável onde será armazenado o dado

Podemos citar como exemplos: leia NOME; leia COR; leia NOTA.

Comando de saída (escreva)

Serve para exibir o resultado do processamento em uma unidade de saída (tela, impressora etc.). É com este comando que podemos exibir uma mensagem qualquer na tela.

Sintaxe:

escreva < argumento >

Onde: escreva : comando de saída
< argumento > : pode ser uma variável ou uma constante (mensagem)

Podemos citar como exemplo: escreva NOME; escreva COR; escreva NOTA.

Encerramento do
Capítulo 3 - Produção de Algoritmos

Quando trabalhamos com a construção de um algoritmo, devemos seguir uma ordem predeterminada. Sendo assim, podemos citar como tipos de estruturas de algoritmos:

Estruturas sequenciais

As estruturas sequenciais são a disposição de um algoritmo, onde sua execução ocorre na sequência na qual os comandos foram descritos, sem nenhum desvio um após o outro.

A estrutura abaixo mostra a implementação de um pseudocódigo de um algoritmo em sequência.

Formato pseudo-código	Formato DNS
INÍCIO	
comando1	comando1
comando2	comando2
...	...
comando n	comando 4
FIM	

Estruturas condicionais

A estrutura de seleção ou decisão se faz necessária quando o uso da estrutura sequencial já não atende as necessidades do usuário, tendo como o exemplo o algoritmo de soma entre dois números.

Como faria para decidir qual dos dois números digitados na entrada de dados é o maior?

Seria necessário um teste que só pode ser implementado utilizando a estrutura de decisão.

Esta estrutura é classificada como simples ou composta conforme a sintaxe a seguir :

Seleção simples
SE < condição> **ENTÃO** < ação> **FIM-SE** OBS.: o FIM-SE informa o final da estrutura

Seleção composta com uma ação apenas
SE < condição> **ENTÃO** < ação 1> **SENÃO** < ação 2> **FIM-SE**

Seleção composta com mais de uma ação
SE < condição> **ENTÃO** < ação 1> < ação 2> < ação 3> < ação n> **SENÃO** < ação 10> < ação 11> < ação 12> < ação n> **FIM-SE**

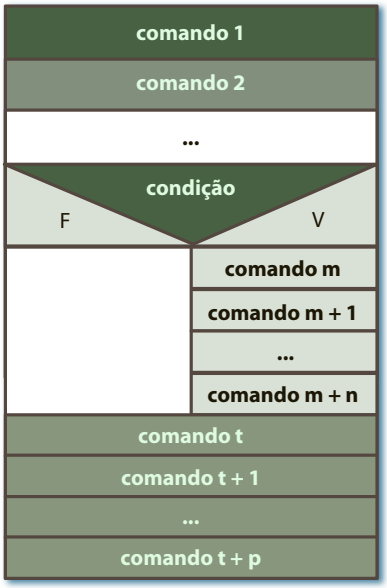
Estruturas condicionais de seleção simples

Quando a execução de um comando (ou grupo de comandos) depende de uma condição verdadeira e não há comandos para executar uma condição falsa.

Formato pseudo-código simples

```
INÍCIO
        comando1
        comando2
        ...
        Se CONDIÇÃO então
        comando m
        comando m+1
        ...
        comando m+n
        fim-se
        comando t
        ...
        comando t+p
FIM
```

Formato DNS



Exemplo: somar dois números, verificando se o primeiro número digitado é maior que o segundo.

Algoritmo: Somar dois números;

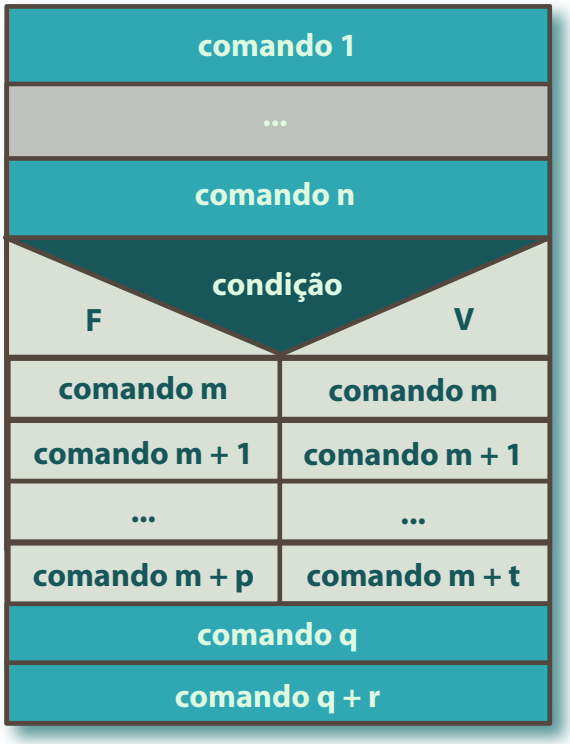
```
Declarações: soma, n1, n2: números;
INÍCIO
        Escreva ("Digite o primeiro número:")
        Leia (n1);
        Escreva ("Digite o segundo número:")
        Leia (n2);
        Soma ← n1 + n2;
        SE n1>n2  então
        Escreva ("N1 é maior que N2")
        FIM-SE
        Escreva ("A soma dos dois números é :", soma)
FIM
```

Estruturas condicionais de seleção composta

Quando se executa um comando (ou grupo de comandos) se uma condição for verdadeira e se executa outro comando (ou grupo de comandos) se uma condição for falsa.

Formato pseudo-código composto

```
INÍCIO
        comando1
        ...
        comando 2
        Se CONDIÇÃO  então
       comando m
       comando m + 1
       ...
       comando m + t
       Senão
        comando m
        comando m + 1
        ...
        comando m + p
       fim-se
        comando q
        ...
        comando q+ r
FIM
```



Exemplo: somar dois números, verificando qual dos dois números digitados é o maior.

Algoritmo: Somar dois números;

```
Declarações: soma, n1, n2: números;
INÍCIO
        Escreva ("Digite o primeiro número:")
        Leia (n1);
        Escreva ("Digite o segundo número:")
        Leia (n2);
        Soma ← n1 + n2;
        SE n1>n2  então
        Escreva ("N1 é maior que N2")
        SENÃO
        Escreva ("N2 é maior que N1)
        FIM-SE
        Escreva ("A soma dos dois números é :", soma)
FIM
```

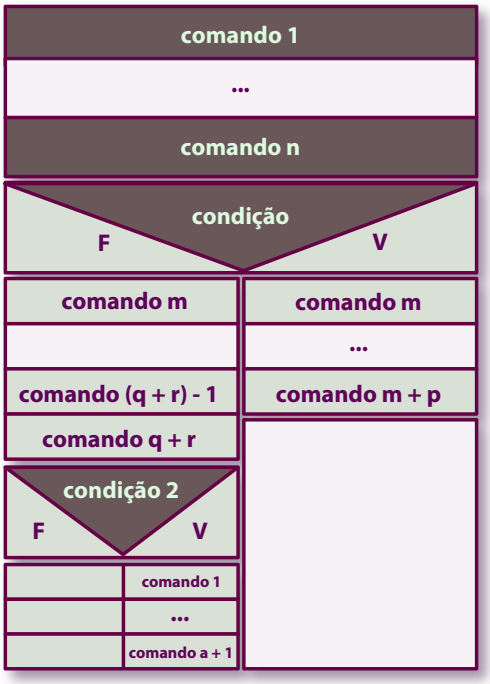
Estruturas condicionais de aninhamento de seleções

O aninhamento de seleções é quando fazemos uso do bloco de comandos de outras estruturas de seleções, sendo utilizada quando se faz necessário o uso de vários testes dentro de uma estrutura.

Formato pseudo-código em seleções

```
INÍCIO
        comando1
        ...
        comando n
        Se CONDIÇÃO      então
       comando m
        ...
        comando m + p
        ...
        comando m + t
        Senão
        comando m
        ...
        comando m+ t

                         Se CONDIÇÃO 2 então
                         comando q
                         ...
                         comando q + r
                         fim-se
        comando s
        ...
        comando s + l
        fim-se
FIM
```



Exemplo: somar dois números, verificando se eles são iguais ou qual dos dois números digitados é o maior.

Algoritmo: Somar dois números;

```
Declarações: soma, n1, n2: números;
INÍCIO
        Escreva ("Digite o primeiro número:")
        Leia (n1);
        Escreva ("Digite o segundo número:")
        Leia (n2);
        Soma ← n1 + n2;
        SE n1 = n2   então
        Escreva ("N1 e N2 são iguais")
        SENÃO
                SE n1 > n2 então
                Escreva ("N1 é maior que N2")
                SENÃO
                Escreva ("N2 é maior que N1")
                FIM-SE
                Escreva ("A soma dos dois números é:", soma)
FIM
```

Estruturas repetitivas

A estrutura de repetição pode ser de dois tipos, controladas (para) e não controladas (enquanto). Estas estruturas são utilizadas quando um conjunto de instruções ou comandos deve ser executado repetidamente, enquanto uma determinada condição permanecer verdadeira. Chamamos isso de laço (***loop***).

Os laços têm as seguintes características:
- Permitem o aninhamento de laços;
- É necessário inicializar a(s) variável(is) que controla(m) o laço antes do início do laço;
- É necessário incrementar a(s) variável(is) que controla(m) o laço dentro do laço.

As repetições podem ser construídas da seguinte forma:
1. Repetição com teste no início;
2. Repetição com teste no final;
3. Repetição com uso de sinalizadores;
4. Repetição controlada.

Estruturas repetitivas com teste no início

A estrutura de repetição ENQUANTO, é utilizada quando se deseja repetir determinadas ações "N" vezes, até que a condição pré-estabelecida não mais seja satisfeita. Essa estrutura se diferencia da estrutura PARA por existir um *FLAG* controlador.

Flag é a variável utilizada na condição da estrutura ENQUANTO a qual receberá valores que influenciarão diretamente no looping (laço de repetição), podendo satisfazer ou não a condição, ou seja, podendo continuar ou não o looping.

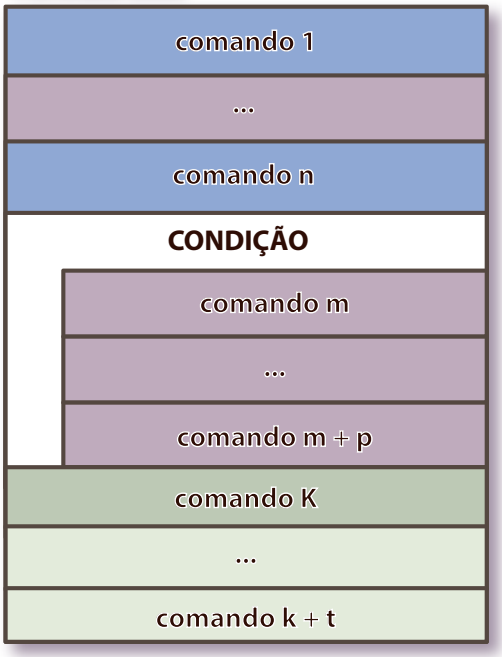
Repetição com teste no início

ENQUANTO <condição> **FAÇA**
<rotina de repetição>
FIM - ENQUANTO

Formato pseudo-código com repetição de teste no início

```
INÍCIO
        comando 1
        ...
        comando n
        ENQUANTO        condição FAÇA
        comando m
        ...
        comando m + p
        FIM-ENQUANTO
        comando k
        ...
        comando k + t
FIM
```

Formato DNS



```
Programa exemplo;
    VAR
        NOME, ENDEREÇO: Literal
INÍCIO
        Leia (NOME);
        { }
        ENQUANTO NOME <>`FIM`, FAÇA
        WRITE (`NOME:`); READLN(NOME)
        WRITE (`Endereço: `); READLN(ENDEREÇO)
        FIM-ENQUANTO
FIM
```

No exemplo acima, utilizamos uma estrutura de repetição ENQUANTO, com um *FLAG* NOME, que funcionará da seguinte maneira:

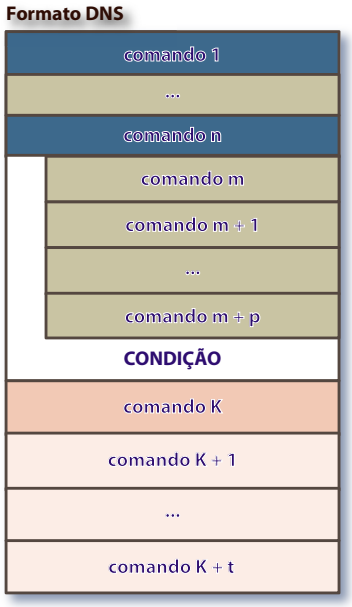
Antes de iniciar a estrutura ENQUANTO, o usuário digitará um NOME. Se, nesse momento, o usuário digitar "FIM", então será finalizado o programa, pois somente entrará na estrutura se a variável NOME for diferente (<>) de "FIM"; caso contrário, entrará na estrutura e, dentro dela, o usuário digitará o ENDEREÇO e, em seguida, digitará novamente o nome (já que é uma estrutura de repetição). Com isso, será mais uma vez verificada a condição e, então, as ações desta estrutura poderão continuar ou não, dependendo do valor assumido pela variável NOME.

Estruturas repetitivas com teste no final

Quando se deseja executar um comando pelo menos uma vez, pode-se fazer o teste no final. Essa validação tem um uso bastante efetivo na validação de dado na entrada pelo teclado, mas sempre pode ser substituída por um enquanto.

A vantagem do Repita... Até é que não é preciso inicializar a(s) variável(is) de controle antes de entrar no laço.

```
Formato pseudo-código com repetição de teste no final
INÍCIO
        comando 1
        ...
        comando n
        REPITA
        comando m
        comando m + 1
        ...
        comando m + p
        ATÉ condição
        comando k
        comando k + 1
        ...
        comando k + t
FIM
```



Exemplo: leia o nome de um funcionário até que no campo nome seja digitado "fim".

```
Algoritmo: leitura de nome;
Declarações:      nome

INÍCIO
        REPITA
        Escreva ("nome:");
        Leia (nome);
        ENQUANTO (nome) <> "fim"
FIM
```

Estruturas repetitivas com uso de sinalizadores

O sinalizador ou *Flag* é um valor que quando lido, sinaliza a saída de um laço, programa ou fim da lista de um arquivo.

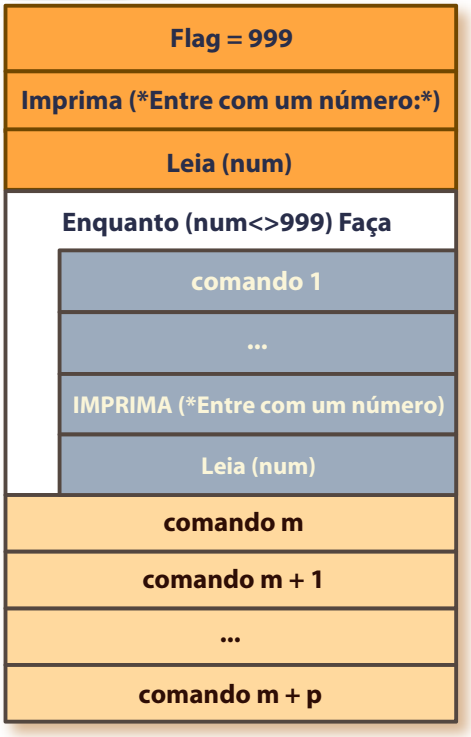
Formato pseudo-código com repetição com uso de sinalizadores

```
Algoritmo XPTO

{Declarações de variáveis}
Const
        Flag=999
Var
        INTEIRO: num
{ corpo do algoritmo}

INÍCIO
        IMPRIMA         ("Entre com um número:")
        LEIA            (num)
        ENQUANTO (num<>flag) FAÇA
        comando 1
        ...
        IMPRIMA         ("Entre com um número:")
        LEIA
        FIM-ENQUANTO
        comando n
        comando m + 1
        ...
        comando m + p
FIM
```

Formato DNS



Estruturas repetitivas com repetição controlada

A estrutura de repetição PARA é utilizada quando se deseja repetir as ações de um algoritmo N vezes. Para isso, precisaremos de uma variável que funcionará como um contador e, quando esse contador atingir o valor numérico desejado, o módulo será finalizado.

Repetição controlada

```
Var X:      inteiro;

INÍCIO

            PARA X ←1 até 10, FAÇA
            <módulo que se deseja repetir>
            FIM-PARA
FIM
```

Programa Exemplo;

```
Declare X:      inteiro;
NOME:           literal
INÍCIO

            PARA X ←1 até 10, FAÇA
            Leia (NOME);
            FIM-PARA
FIM
```

No exemplo acima, temos um módulo que é tão somente para ler um NOME, porém como esse módulo está dentro de uma estrutura de repetição e essa estrutura obriga que seja repetido 10 (dez) vezes (para X de 1 até 10), então serão lidos 10 nomes, ou seja, o usuário digitará 10 nomes.

Após digitar todos, abandonará o tópico (Fim-Para) e irá para a próxima instrução abaixo que é tão somente Fim, com isso finalizará o programa.

Encerramento do
Capítulo 4 - Estruturas de Algoritmos

A modularização de algoritmos visa subdividir o programa de modo a facilitar e permitir o entendimento, permitir a reutilização e evitar a repetição desnecessária de blocos do programa.

Estudaremos dois tipos básicos de modularização, a saber:

Procedimentos (Procedures)

Um procedimento é uma sequência de comandos que possa realizar qualquer tipo de procedimento que o programador desejar.

As variáveis podem ser locais ou globais.

As variáveis locais são tipos, constantes declaradas dentro do procedimento que só são acessíveis dentro dos comandos do procedimento.

JJá as variáveis globais são tipos e constantes declaradas logo após o identificador do algoritmo, que são acessíveis e visíveis dentro de qualquer procedimento. No algoritmo as variáveis são globais.

```
Procedimento IDENTIFICADOR_PROC (Lista de parâmetros)
{Breve descrição do que faz}
Const
(...)
Tipo
(...)
Var
(...)
INICIO
(...)
FIM IDENTIFICADOR_PROC
```

Funções (Functions)

Uma função é um módulo que tem por objetivo calcular um valor ou realizar qualquer outra operação e retornar ao algoritmo “chamador” um valor.

Ao ser invocada uma função a mesma retorna um valor para o seu chamador. Por exemplo, quando chamamos uma função de validação de CPF enviando o número do CPF, temos como retorno uma variável dizendo se o CPF é ou não válido.

A chamada de uma função para execução é feita apenas citando o seu identificador no meio de uma expressão.

```
FUNÇÃO IDENTIFICADOR_FUNCAO (Lista de parâmetros)
{Breve descrição do que faz a função}
Const
(...)
Tipo
(...)
Var
(...)
INICIO
(...)
IDENTIFICADOR_FUNCAO <- <EXPRESSAO>
FIM
FIM FUNÇÃO IDENTIFICADOR_FUNCAO
```

Encerramento do
Capítulo 5 - Modularização de Algoritmos

Você sabe dizer o que é uma recursão? E o que significa recursividade?

A recursão é uma técnica que define um problema em termos de uma ou mais versões menores deste mesmo problema. Essa ferramenta pode ser utilizada sempre que for possível expressar a solução de um problema em função do próprio problema.

A recursividade (ou recursão) é encontrada principalmente na matemática, mas está presente em algumas situações do cotidiano. Por exemplo, quando um objeto é colocado entre dois espelhos planos paralelos e frente a frente surge uma imagem recursiva, porque a imagem do objeto refletida num espelho passa a ser o objeto a ser refletido no outro espelho e, assim, sucessivamente.

Em programação, a recursividade é um mecanismo útil e poderoso que permite a uma função chamar a si mesma direta ou indiretamente, ou seja, uma função é dita recursiva se ela contém, pelo menos, uma chamada explícita ou implícita a si própria.

A ideia básica de um algoritmo recursivo consiste em diminuir sucessivamente o problema até que o tamanho ou a simplicidade do problema reduzido permita resolvê-lo de forma direta, sem recorrer a si mesmo.

Quando isso ocorre, diz-se que o algoritmo atingiu uma condição de parada, a qual deve estar presente em pelo menos um local dentro algoritmo.

Sem essa condição, o algoritmo não para de chamar a si mesmo, até estourar a capacidade da pilha, o que geralmente causa efeitos colaterais e até mesmo o término indesejável do programa.

Para todo algoritmo recursivo existe outro correspondente iterativo (não recursivo) que executa a mesma tarefa. Implementar um algoritmo recursivo, partindo de uma definição recursiva do problema, em uma linguagem de programação de alto nível como Pascal e C é simples e quase imediato, pois o seu código é praticamente transcrito para a sintaxe da linguagem.

Por essa razão, em geral, os algoritmos recursivos possuem código mais claro (legível) e mais compacto do que os correspondentes iterativos. Além disso, muitas vezes, é evidente a natureza recursiva do problema a ser resolvido, como é o caso de problemas envolvendo árvores - estruturas de dados naturalmente recursivas.

Entretanto, também há desvantagens:

1. Algoritmos recursivos quase sempre consomem mais recursos (especialmente memória, devido uso intensivo da pilha) do computador, logo tendem a apresentar um desempenho inferior aos iterativos;

2 Algoritmos recursivos são mais difíceis de serem depurados, especialmente quando for alta a profundidade de recursão (número máximo de chamadas simultâneas).

Função fatorial

A função fatorial (!) é um dos exemplos clássicos de recursividade e, por isso, de citação quase obrigatória. Eis sua definição recursiva:

$$n! = \begin{cases} 1 & \text{Se } n = 0 \\ n \,.\, (n-1)! & \text{Se } n>0 \end{cases}$$

Veja como é fácil transformar a definição da função para Portugol e C:

IMPLEMENTAÇÕES RECURSIVAS DA FUNÇÃO FATORIAL	
Portugol	**Linguagem C**

função Fat (n: natural): natural
INÍCIO
  SE n = 0 **então**
  Retorne 1
  SENÃO
  Retorne n* Fat (n-1)
 	
FIM
	unsigned int Fat (unsigned int n)
	{
	  if(n==0)
	  Return 1;
	  else
	  Return n*Fat(n-1);
	}

Pela definição, valor de 4! é calculado como:
4!=4*3!=4*(3*2!)=4*(3*(2*1!))=4*(3*(2*(1*0!)))=4*(3*(2*(1*1)))=24

Note que função é chamada recursivamente com argumento decrescente até chegar ao caso trivial (0!), cujo valor é 1. Este caso trivial (condição de parada) encerra a sequência de chamadas recursivas. A sequência de chamadas é melhor ilustrada abaixo:

```
4! = 4 * 3!
        3! = 3 * 2!
                2! = 2 * 1!                O contexto de cada chamada é empilhado.
                        1! = 1 * 0!
                0! = 1                     Condição de parada
                        1! = 1
                2! = 2                     O contexto de cada chamada é desempilhado.
        3! = 6
4! = 24
```

Função fatorial

Como n!= nX(n-1)X(n-2)X···X3X2X1, é muito simples implementar um algoritmo iterativo da função fatorial.

Abaixo, podemos observar a apresentação de dois algoritmos iterativos que se equivalem:

IMPLEMENTAÇÕES ITERATIVAS DA FUNÇÃO FATORIAL

Portugol

```
função Fat_NR (n: natural): natural
declare
i, x: natural
INÍCIO
        x←1
        PARA i←2até 2 FAÇA
                x←x*i
        RETORNE x
FIM
```

Linguagem C

```
unsigned int Fat_NR (unsigned int n)
{
        unsigned int i, x;
                x=1;
        for (i=2; i<=n; i ++)
                x=x*i
        return x
}
```

Viu como é fácil transformar a definição da função fatorial, para Portugol e C, seja em implementação iterativa ou recursiva?

Vamos comparar as duas estruturas abaixo:

Implementações da função fatorial

IMPLEMENTAÇÕES RECURSIVAS DA FUNÇÃO FATORIAL

Portugol

```
função Fat (n: natural): natural
INÍCIO
        SE=0 então
                Retorne 1
        SENÃO
                Retorne n* Fat (n-1)
FIM
```

Linguagem C

```
unsigned int Fat (unsigned int n)
{
        if (n==0)
                Return 1;
        else
                Return n* Fat (n-1)
}
```

IMPLEMENTAÇÕES ITERATIVAS DA FUNÇÃO FATORIAL

Portugol

```
função Fat_NR (n: natural): natural
declare
i, x: natural
INÍCIO
        x←1
        PARA i←2 até 2 FAÇA
                x←x * i
        RETORNE x
FIM
```

Linguagem C

```
unsigned int Fat (unsigned int n)
{
        unsigned int i, x;
                x=1;
        for (i = 2; i <=n; i++)
                x = x*i;
        return x;
}
```

Sequência de Fibonacci

A sequência [0, 1, 1, 2, 3, 5, 8, 13, 21, ...] é conhecida como sequência ou série de Fibonacci e tem aplicações teóricas e práticas, na medida em que alguns padrões na natureza parecem segui-la. Pode ser obtida por meio da definição recursiva:

$$\text{Fib}(n) = \begin{cases} 0 & \text{Se } n = 0 \\ 1 & \text{Se } n = 1 \\ \text{Fib}(n-1) + \text{Fib}(n-2) & \text{Se } n > 0 \end{cases}$$

A qual pode ser implementada como:

```
FUNÇÃO
função Fib (n: natural): natural
INÍCIO

        SE n = 0 ou n = 1  ENTÃO
                RETORNE n
        SENÃO
                RETORNE Fib (n-2) + Fib (n-1)

FIM
```

Note que, para n > 1, cada chamada causa 2 novas chamadas de Fib, isto é, o número total de chamadas cresce exponencialmente. Observe no diagrama de execução ao lado que para Fib(5), são feitas 14 chamadas da função.

Além disso, Fib(0) e Fib(2) são chamadas 3 vezes; Fib(1) é chamada 5 vezes. Para Fib(25) são feitas 242784 chamadas recursivas!

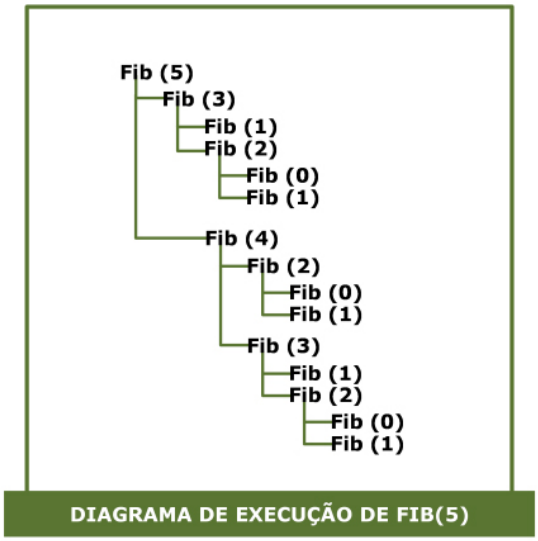


DIAGRAMA DE EXECUÇÃO DE FIB(5)

No caso da sequência de Fibonacci é relativamente simples implementar um algoritmo iterativo com complexidade O(n), que tire proveito dos valores já calculados.

Eis abaixo uma possibilidade.

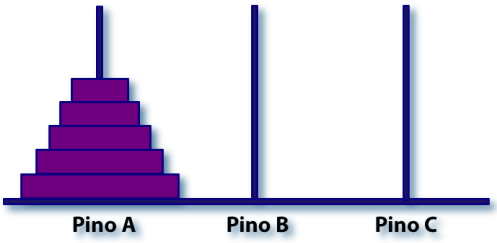
```
FUNÇÃO
função Fib_NR (n: natural): natural
declare
        i, F1, F2, F: natural
INÍCIO
        SE n = 0 ou n = 1 ENTÃO
                RETORNE 1
        SENÃO
        INÍCIO
                F1 ← 0
                F2 ← 1
                PARA i ¬ 1 até -1 FAÇA
                        INÍCIO
                                F1 ← F1 + F2
                                F1 ← F2
                                F2 ← F
                        FIM
                RETORNE F
        FIM
FIM
```

Problema da Torre de Hanói

O problema ou quebra-cabeça conhecido como Torre de Hanói foi publicado em 1883 pelo matemático francês Edouard Lucas, também conhecido por seus estudos com a série Fibonacci.

> Consiste em transferir, com o menor número de movimentos, a torre composta por N discos do pino A (origem) para o pino C (destino), utilizando o pino B como auxiliar. Somente um disco pode ser movimentado de cada vez e um disco não pode ser colocado sobre outro disco de menor diâmetro.

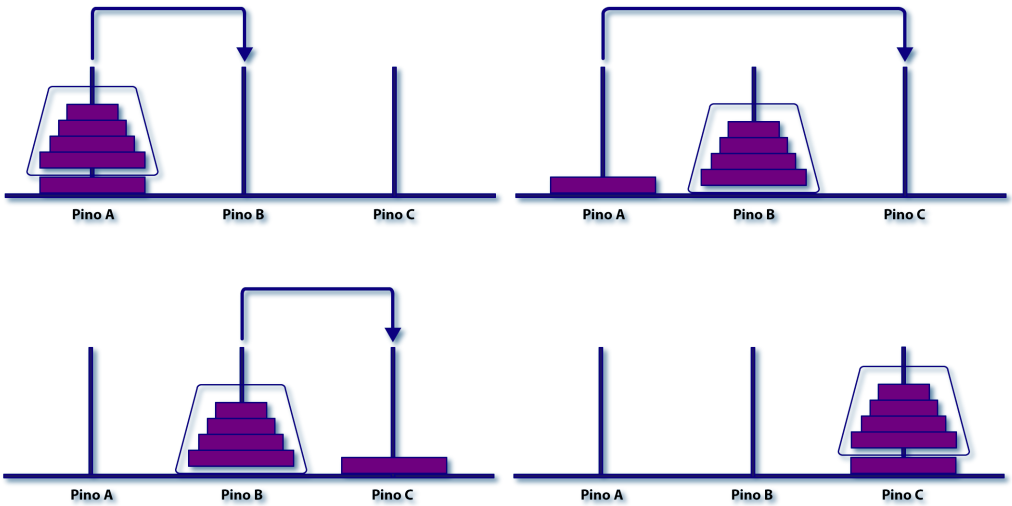
Torre de Hanói



Qual seria a solução desse problema?

Solução: Transferir a torre com N-1 discos de A para B, mover o maior disco de A para C e transferir a torre com N-1 de B para C. Embora não seja possível transferir a torre com N-1 de uma só vez, o problema torna-se mais simples: mover um disco e transferir duas torres com N-2 discos. Assim, cada transferência de torre implica em mover um disco e transferir de duas torres com um disco a menos e isso deve ser feito até que torre consista de um único disco.

Solução do problema da Torre de Hanói



Problema da Torre de Hanói

O algoritmo que soluciona o problema é surpreendentemente simples.

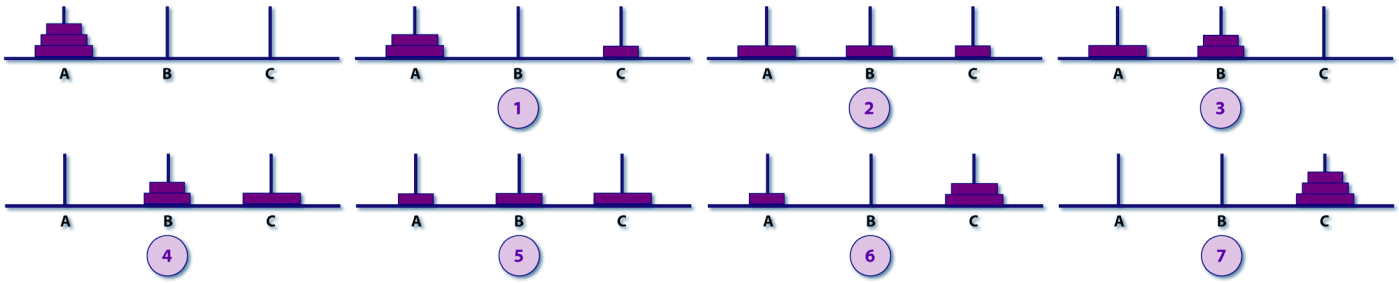
Uma chamada a MoveTorre(3, 'A', 'C', 'B') produziria a seguinte saída:

Movimento: A -> C
Movimento: A -> B
Movimento: C -> B
Movimento: A -> C
Movimento: B -> A
Movimento: B -> C
Movimento: A -> C

Algoritmo

```
Procedimento MoveTorre (N: natural; Orig, Dest, Aux: caracter)
INÍCIO
    SE n=1 ENTÂO
        MoveDisco (Orig, Dest) SENÂO
    INÍCIO
        MoveTorre (N-1, Orig, Aux, Dest)
        MoveDisco (Orig, Dest)MoveTorre (N-1, Aux, Dest, Orig)
    FIM
FIM
Procedimento MoveDisco (Orig, Dest: caracter)
        INÍCIO
            Escreva (Movimento: " , Orig, " →", Dest)
FIM
```

Sequência de movimentos para solucionar problema da Torre de Hanói com 3 discos




```
MT (3, A, C, B)
  MT (2, A, B, C)
    MT (1, A, C, B)
      MD (A, C) ---- A → C
    MD (A, B) --------- A → B
    MT (1, C, B, A)
      MD (C, B) ---- C → B
  MD (A, C) -------- A → C
  MT (2, B, C, A)
    MT (1, B, A, C)
      MD (B, A) -- B → A
    MD (B, C) ------- B → C
    MT (1, A, C, B)
      MD (A, C) -- A → C
```

DIAGRAMA DE EXECUÇÃO DA CHAMADA MOVE TORRE (3, 'A','C','B')

Observe que o algoritmo MoveTorre faz duas chamadas a si mesmo, portanto o número de movimentos cresce exponencialmente com o número de discos. Mais precisamente, a solução do problema com N discos requer 2N-1 movimentos.

Assim, se uma pessoa se dispusesse a resolver o problema com 25 discos, com dedicação exclusiva de 8 horas por dia e realizando um movimento.

Por segundo, esta "pobre criatura" gastaria 3354431 segundos, ou seja, mais de 3 anos para solucionar o problema.

Esse tempo dobraria a cada disco acrescentado!

Quais são as vantagens, desvantagens e aplicações desse tipo de algoritmo?

APLICAÇÕES

Nem sempre a natureza recursiva do problema garante que um algoritmo recursivo seja a melhor opção para resolvê-lo. O algoritmo recursivo para obter a sequência de Fibonacci é um ótimo exemplo disso.

VANTAGENS

Os algoritmos recursivos normalmente são mais compactos, mais legíveis e mais fáceis de serem compreendidos. Algoritmos para resolver problemas de natureza recursiva são fáceis de serem implementados em linguagens de programação de alto nível.

DESVANTAGENS

Por usarem intensivamente a pilha, o que requer alocações e desalocações de memória, os algoritmos recursivos tendem a ser mais lentos que os equivalentes iterativos, porém pode valer a pena sacrificar a eficiência em benefício da clareza. Algoritmos recursivos são mais difíceis de serem depurados durante a fase de desenvolvimento.

Algoritmos recursivos são aplicados em diversas situações como em:

- Problemas envolvendo manipulações de árvores
- Analisadores léxicos recursivos de compiladores
- Problemas que envolvem tentativa e erro ("Backtracking")

Encerramento do Capítulo 6 - Recursividade

Definimos uma estrutura de dados como um conjunto de dados relacionados entre si, agrupados na memória principal ou secundária. As estruturas de dados existem para facilitar a manipulação dos dados pelo programador quando estes se tornarem mais complexos.

Vamos conhecer todas as principais estruturas, mas neste curso, estudaremos apenas registros, vetores e matrizes.

- Listas Lineares (*Linked Lists*)
- Vetores (*Arrays)* e Matrizes ou agregados homogênios
- Árvores (*Trees*)
- Filas (*Queue*)
- Pilhas (*Stacks*)
- Registros (*Records*) ou agregados heterogênios

Estrutura de dados homogênea

Para trabalharmos com dados homogênios (vetores e matrizes), precisamos estudar algumas estruturas de controle.

Repetição com variável de controle (variando)

É mais um tipo de estrutura com repetição, sendo a diferença dessa estrutura em relação às anteriores - Repita e Enquanto - é que é necessário saber antecipadamente o número exato de vezes que o laço deve ser repetido.

A vantagem do comando Variando é que ele dispensa a inicialização e a modificação da variável de controle do laço, uma vez que ela é inicializada e modificada automaticamente.

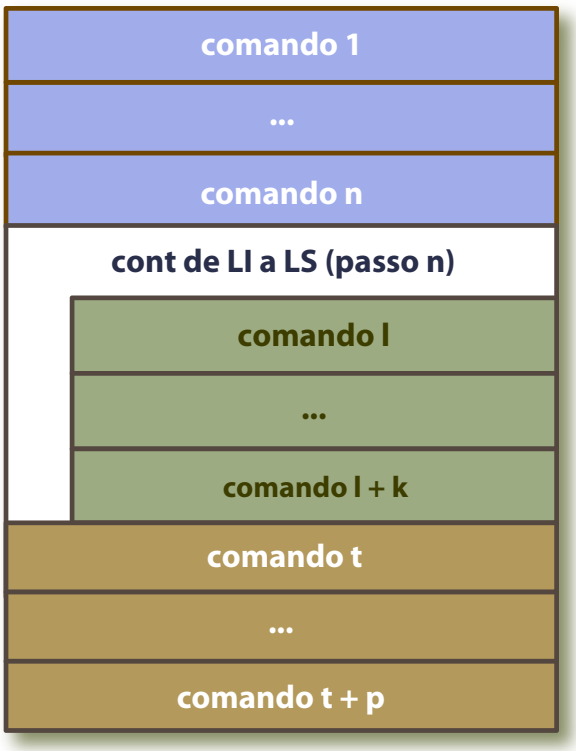
Formatos pseudocódigo e DNS

Formato pseudo-código com variável de controle

```
ALGORITMO: XPTO
{Declaração de variáveis}
Var

        INTEIRO: cont
{Corpo do algoritimo}

INÍCIO
        comando 1
        ...
        comando n
        Variando cont de LI a LS, [passon] FAÇA
                comando I
                ...
                comando I + k
        fim variando
        comando t
        ...
        comando t + p
FIM
```



- **Cont:** é a variável de controle do laço.
- **LI:** limite inferior da variável de controle.
- **LS:** limite superior da variável de controle.
- **Passo n:** intervalo entre dois valores subsequentes de n. É opcional quando o incremento é 1.

Faça um algoritmo que escreva os números pares ente 0 e 100.

```
Algoritmo: números pares;
Var nun_par:    inteiro;
INÍCIO
                PARA nun_par de 0 até 100 passo 2 FAÇA
                        Escreva (" o número par é:", nun_par)
                FIM-PARA
FIM
```

Seleção dentre muitas alternativas

É a opção de aninhamentos de seleção.

É indicado o uso quando a variável ou expressão puder assumir diferentes valores.

Formatos pseudocódigo e DNS

Formato pseudo-código em seleção múltipla

```
INÍCIO
        comando 1
        ...
        comando n
        Conforme
                Caso    IDENTIFICADOR = valor 1
                        comando k
                        ...
                        comando k + l
                Caso    IDENTIFICADOR = valor 2
                        comando k
                        ...
                        comando k + p
                Caso    IDENTIFICADOR = valor 3
                        comando k
                        ...
                        comando k + q
                Senão
                        comando k
                        ...
                        comando k + o
        fim conforme
        comando s
        ...
        comando s + w
FIM
```

comando 1			
...			
comando n			
IDENTIFICADOR / F	Valor 1	Valor 2	Valor 3
comando k		comando k	comando k
...	...	...	...
comando k + o	comando k + lc	omando k + p	comando k + q
comando s			
...			
comando s + w			

Leia a idade de um pessoa e diga se ela é: uma criança - até 10 anos, de 11 á 18 - pré-adolescente, de 18 a 25 - adolescente, de 26 a 64 - adulto e de 65 em diante - idoso.

```
Algoritmo: idade;

Var idade: inteiro
INÍCIO
        Escreva ("idade:")
        Leia    ("idade:");
        Conforme
                Caso idade < = 10
                Escreva "Criança"
                Caso idade > 10 and idade <=18
                Escreva "Pré-adolescente"
                Caso idade> 18 and idade <= 25
                Escreva "Adolescente"
                Caso idade > 26 and idade <=64
                Escreva "Adulto"
                Senão
                Escreva "Idoso"
        Fim conforme
FIM
```

Estrutura de dados homogênea - Registros

Os registros são conhecidos também como dados agregados heterogênios ou compostos.

Eles são um conjunto de informações relacionadas entre si que são compostos, normalmente, de informações de diferentes tipos. Por isso, podemos dizer que um registro é formado por N campos, tantos quantos o programador desejar.

```
TIPO

IDENTIFICADOR DO REGISTRO = REGISTRO
        TIPO DE CAMPO 1: IDENTIFICADOR DO CAMPO 1
        TIPO DE CAMPO 2: IDENTIFICADOR DO CAMPO 2
        ...
        TIPO DE CAMPO N: IDENTIFICADOR DO CAMPO N
FIM IDENTIFICADOR DO REGISTRO
```

Formato pseudo-código de registro

```
TIPO
        REGALUNO: REGISTRO
                TEXTO: nome
                CARACTERE: sexo
                INTEIRO: cidade
                REAL: Média
        FIM REGALUNO
        REGENDEREÇO: REGISTRO
                TEXTO: rua
                INTEIRO: número
                TEXTO: complemento
                TEXTO: bairro, cidade, estado
                INTEIRO: cep, extCEP
        FIM REGENDEREÇO
        REGCAD: REGISTRO
                TEXTO: Nome, Departamento
                REGEENDERÇO: Endereço
                INTEIRO: Telefone
                REAL: Salário
        FIM REGECAD
        VAR
            REGALUNO: RAluno
            REGCAD: Rfunc, MelhorFunc
            REGALUNO: RAlunoA, RAlunoB
```

Estrutura de dados heterogênea - Trabalhando com registros

Os registros podem ser acessados como um todo.

Leia (RAluno)

```
{Lê o dispositivo de entrada e armazena em RAlunoA}
RalunoB ← RAlunoA
{O conteúdo RAlunoA é copiado campo a campo para RAlunoB}
INÍCIO
        IMPRIMA     (" Entre com os dados do aluno")
        LEIA        (RAlunoA)
        IMPRIMA     ("Nome do aluno):", RAlunoA.nome)
        IMPRIMA     (" Sexo do aluno:" , RalunoA.sexo)
        IMPRIMA     ("Cidade do aluno:", RAlunoA.cidade)
        IMPRIMA     ("Média do aluno:" RAlunoA.média)
FIM
INÍCIO
        IMPRIMA     ("Entre com os dados do funcionário")
        LEIA        (RFunc)
        IMPRIMA     ("Nome do funcionário:", RFunc.nome)
        IMPRIMA
                    ("Endereço do funcionário:", RFunc.endereço.rua, RFunc.enedereço.número,
                    RFunc.endereço.complemento, RFunc.endereço.bairro, RFunc.endereço.cidade,
                    RFunc.endereço.estado, RFunc.endereço.cep, RFunc.endereço.ExtCEP)
FIM
```

Estrutura de dados heterogênea - Arquivos

É um conjunto de dados de tamanho limitado que é identificado por um nome (Identificador e Extensão) que está localizado na memória secundária.

```
var
ARQUIVO DO TIPO: Identificador do Arquivo
```

```
Var
        ARQUIVO DE INTEIRO: ARQNUM1, ARQNUM2
        ARQUIVO DE TEXTO: ARQNOME
        ARQUIVO DE REAL: ARQMEDIA
        ARQUIVO DE REGFUNC: ARQFUNC
        ARQUIVO DE REGALUNOS: ARQALUNO
        INTEIRO: Num
INÍCIO
        IMPRIMA     ("Lendo 1º registro do arquivo ARQNUM1:")
        LEIA        (ARQNUM1, Num)
        IMPRIMA     ("Gravando no arquivo ARQNUM2:")
        GRAVE       (ARQNUM2, Num)
FIM
```

Encerramento do
Capítulo 7 - Estrutura de dados

Linguagens estruturadas

Nesta classe, encaixam-se as chamadas linguagens de programação de alto nível surgidas em meados dos anos 1960. As linguagens concebidas neste período foram resultado da necessidade da produção de código de programa de forma clara, aparecendo o conceito de estruturação do código (endentação, utilização de letras maiúsculas e minúsculas nos identificadores, eliminação de instruções "problemáticas" como o go to etc...).

O período compreendido entre a década de 1960 e a de 1980 foi bastante produtivo no que diz respeito ao surgimento de linguagens de programação, o que permitiu o aparecimento de uma grande quantidade de linguagens as quais podem ser organizadas da seguinte forma:

Linguagens de uso geral

Podem ser utilizadas para implementação de programas com as mais diversas características e independentemente da área de aplicação considerada; encaixam-se nesta categoria linguagens como **Pascal**, **Modula-2** e **C**.

Linguagens especializadas

São orientadas ao desenvolvimento de aplicações específicas; algumas das linguagens que ilustram esta categoria são ***Prolog***, ***Lisp*** e ***Forth***.

Linguagens orientadas a objeto

Oferecem mecanismos sintáticos e semânticos de suporte aos conceitos da programação orientada a objetos; alguns exemplos destas linguagens são *Smalltalk*, ***Eiffel***, **C++** e ***Delphi***.

Encerramento do
Capítulo 8 - Linguagens Estruturas

Nessa unidade você estudou os seguintes assuntos:

- Introdução à lógica de programação;
- Definição e produção de algoritmos;
- Técnicas de representação de algoritmos;
- Estruturas dos algoritmos e de dados;
- Modularização de algoritmos;
- Recursividade;
- Linguagens estruturadas.

Conclusão

Com esse conhecimento, você terá a capacidade de solucionar problemas de navegação através de algoritmos, transformar layout em código, desenvolver lógica de programação para solução de problemas e codificar soluções utilizando uma linguagem estruturada, objetivando a publicação do site informativo.

Parabéns, você concluiu a unidade de Lógica de Programação e Estrutura de dados